

For very large datasets, replication is not enough, we need break the data up into **partitions** aka **sharding**

Partitions defined in such a way that each piece of data (row, record, doc) **belongs to exactly one partition**

↓ various ways to achieve this

Why partition? → **scalability**



different partitions placed on dif. nodes ∴ large dataset can be distributed across many disks, and **query load can be distributed across many processors**

Partitioning and Replication:

- usually done together for fault tolerance
- nodes may store more than one partition (ex. Node_1 is a leader for some partition, and a follower of some different partitions)

Partitioning of Key-Value Data:

Goal of partitioning is to spread data and query load evenly across nodes — In theory 10 nodes should be able to handle 10x data and r/w throughput of 1 node

If partitions are unfair (some have more data/queries than others) we call the partition **skewed**

A partition with a disproportionately high load is called a **hot spot** (usually the bottleneck)

Lets assume we have a simple key-value data model where you always access a record by its primary key

One way of partitioning: **Partitioning by Key Range**

Assign a continuous range of keys (from some min to max) to each partition, like chapters in a dictionary

- if you know boundaries between ranges, you can easily determine which partition contains the key
- Can keep keys in sorted order within each partition (LSM Trees),

making range scans easy

- make sure your partitions adapt to the data to ensure equal distribution of data

↳ ex. partitioning for each letter in alphabet; A partition will have much more data/queries than Z partition

Another way of partitioning: Partitioning by Hash of Key

To reduce risk of skew and hotspots, some datastores use hash functions to partition

A good hash function takes skewed data and uniformly distributes it

Assign partitions a range of hashes (rather than keys)

- lose the property of being able to do efficient range queries

Cassandra has found a compromise between the two methods using a

compound primary key consisting of several columns

↳

ex. on social media site where compound PK = (user_id, update_timestamp)
only first part of key (user_id) is hashed to determine partition, and other columns are used sort the data: efficiently retrieve all updates made by some user within some time interval, sorted by timestamp

Skewed Workloads and Relieving Hot Spots:

Hashing can reduce skew/hot spots but cannot avoid them entirely:

- Data partitioned by Key
- All r/w for a key go to the same partition
- If one key gets massive traffic, that partition becomes a hot spot

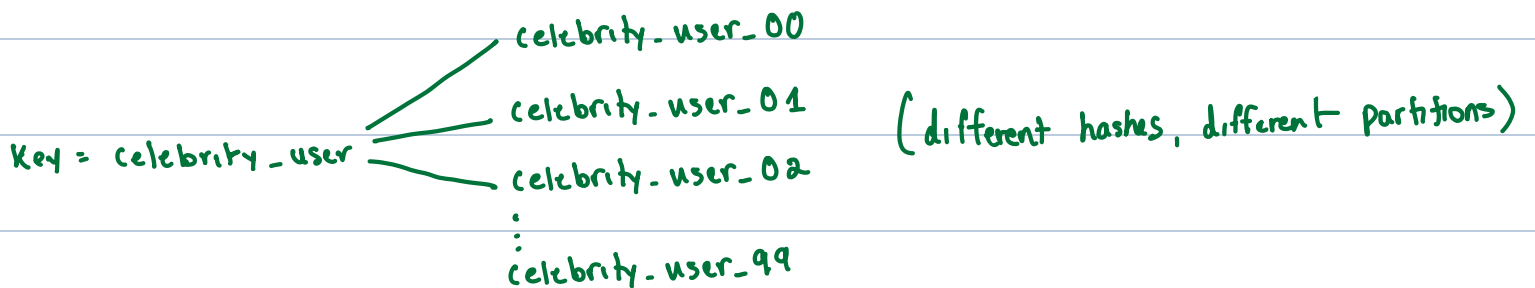
Responsibility of the application to reduce the skew —

Ex of a simple technique:

If a key is known to be hot,

lets say: Key = celebrity-user

you can append/prepend a random number to split the writes,



Now instead of 100% of traffic to 1 partition, its $\sim 1\%$ of traffic to 100 partitions

However:

- any reads to this key now must aggregate data from all split keys

Partitioning and Secondary Indexes:

Partitioning schemes discussed so far work if records are only ever accessed using their PK; situation becomes more complicated if secondary indexes are involved.

↓
usually doesn't uniquely identify a record; used to search for occurrences of a particular value

Two main approaches to partitioning with secondary indexes:

- 1) **Document-based partitioning** (MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB)
 - aka **local index**

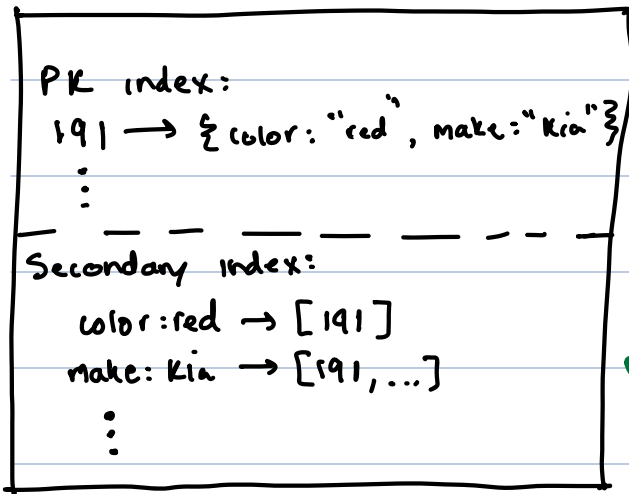
In this approach, each partition is completely separate and maintains its own secondary indexes, covering only documents in that partition

↓
if you have declared the index, the database will perform the indexing automatically

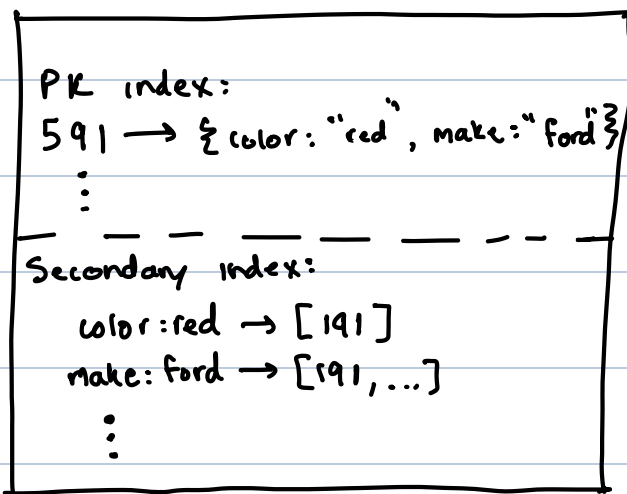
For queries on secondary indexes, need to send the query to **all partitions** and aggregate the results → known as **scatter/gather** and can make reads expensive (prone to tail latency amplification)

Ex. You have two partitions, partition 1 holds documents with IDs 0 to 499, partition 2 holds documents with IDs 500 to 999

Partition 1



Partition 2



(Scatter/gather)



I want red car

2) Term-based Partitioning

- aka global index

Construct a global index that covers data in all partitions;

This global index must also be partitioned, but it can be

partitioned differently than the primary index



Partition 1

PK index:
191 → { color: "red", make: "bmw" }
⋮

Secondary index:
color: red → [191, 591]
make: yamaha → []
⋮

Partition 2

PK index:
591 → { color: "red", make: "bmw" }
⋮

Secondary index:
color: blue → []
color: green → []
make: bmw → [191, 591]
⋮

In this example, red cars from all partitions exist in partition 0 because the global index is partitioned so that :

Partition 1 :

- holds colors starting with the letters r to z
- holds makes starting with the letters g to z

Partition 2 :

- holds colors starting with the letters a to q
- holds makes starting with the letters a to f

We call this **term-partitioned** because partition we query depends on the term we are looking for

Reads are more efficient, but writes are slower and more complicated compared to document based

Rebalancing Partitions :

Process of moving load from one node in the cluster to another is called **rebalancing**

Rebalancing minimum requirements :

- after rebalancing, load should be shared fairly between nodes in the cluster
- DB should continue accepting r/w while rebalancing
- No more data than necessary should be moved to make it faster

Strategies :

1) **Fixed partitions** (Riak, ElasticSearch, Couchbase, Voldemort)

- create many more partitions than number of nodes and assign each node several partitions
- if node added to cluster it can steal partitions from other nodes ; reverse if node is removed
- only thing that is changed is the assignment of partitions to nodes
- Choose the right number of partitions in comparison to node count. too high or low is bad

2) Dynamic Partitioning (HBase, MongoDB)

- each partition assigned to one node, each node can handle multiple partitions
- When a partition grows to exceed a preconfigured size, it will split so that about $\frac{1}{2}$ of the data ends up on each side; one of the two halves can be sent to another node
- Conversely, if data shrinks below some threshold it can be merged with an adjacent partition
- **Caveat:** Until it hits the point at which first partition is split, all writes must be processed by one node; can do **pre-splitting** to counteract this.

3) Partitioning proportionally to nodes (Cassandra, Ketama)

- have a fixed number of partitions per node
- when adding a new node, this node randomly chooses a fixed number of partitions to split and takes ownership of one half of each of those split partitions

Good to have a human in the loop for rebalancing. It's slower but can help prevent operational surprises

Request Routing

Something needs to stay on top of changes in order to answer the question, "If I want to r/w the key "foo", which IP address and port number do I need to connect to?"

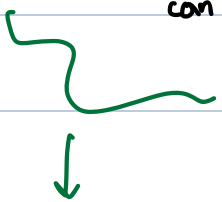


This is an instance of a more general problem called

Service discovery

Three high level approaches to the problem:

1. Clients contact any node (round-robin load balancer), node handles the request if it can (coincidentally owns partition needed) otherwise forwards request to appropriate node, receives reply, and passes reply to client
2. All requests pass through load balancer which determines what node each request goes through
3. Require that clients be aware of the partitioning so client can connect directly



Key Problem: how does the component making the routing decision (client, node, load balancer) learn about changes in assignments

Answer: Use a coordination service ex. Zookeeper

↓
used by: HBase, Elasticsearch, Kafka

Cassandra uses **gossip protocol** among nodes to disseminate changes in cluster state ∴ Requests can be made to any node (approach 1)